



Universidad Carlos III
Curso 2024-25

Criptografía y Seguridad Informática

Desarrollo de una aplicación que utiliza
criptografía

Grupo: 81, ID de grupo de prácticas: 02, [Github](#)

Luca Petidier Iglesias , 100496633,
100496633@alumnos.uc3m.es
Marcos Romo Poveda, 100496072,
100496072@alumnos.uc3m.es

ÍNDICE:

ÍNDICE:	2
Aplicación Web:	3
1. ¿Cuál es el propósito de su aplicación?.....	3
2. ¿Cuál es su estructura interna?.....	3
Firma:	4
1. ¿Para qué se utiliza la firma digital?.....	4
2. ¿Qué algoritmos ha utilizado y por qué?.....	4
3. ¿Cómo se gestionan y almacenan las claves y las firmas?.....	4
Certificados:	5
1. ¿Cómo se generan los certificados de clave pública?.....	5
2. ¿Qué jerarquía de autoridades de certificación se ha desplegado?.....	5
3. ¿Por qué ha escogido esta configuración y no otra?.....	5
4. ¿Cómo se ha implementado?.....	5
5. ¿En qué momento se utilizan los certificados y para qué?.....	5
Complejidad y diseño:	6
Mejoras:	7
Declaración de uso de IA:	8

Aplicación Web:

1. ¿Cuál es el propósito de su aplicación?

En esta primera entrega, hemos decidido crear una aplicación de intercambio de mensajes entre personas (no a uno mismo), con la posibilidad de añadir más funciones en el futuro.

Para hacer que sea más cómodo para el usuario, hemos integrado Python con HTML y CSS, de modo que, además de cumplir con su función principal, la aplicación sea más fácil de usar y navegar.

Aunque aún no está implementado, para la segunda entrega planeamos añadir roles de usuario. Los roles de “Admin” y “Técnico” podrán realizar las funciones básicas de los usuarios, como el envío de mensajes, pero también podrán eliminar mensajes ofensivos y vetar a aquellos usuarios que envíen demasiados mensajes inapropiados.

2. ¿Cuál es su estructura interna?

Internamente nuestra aplicación se divide en varias secciones:

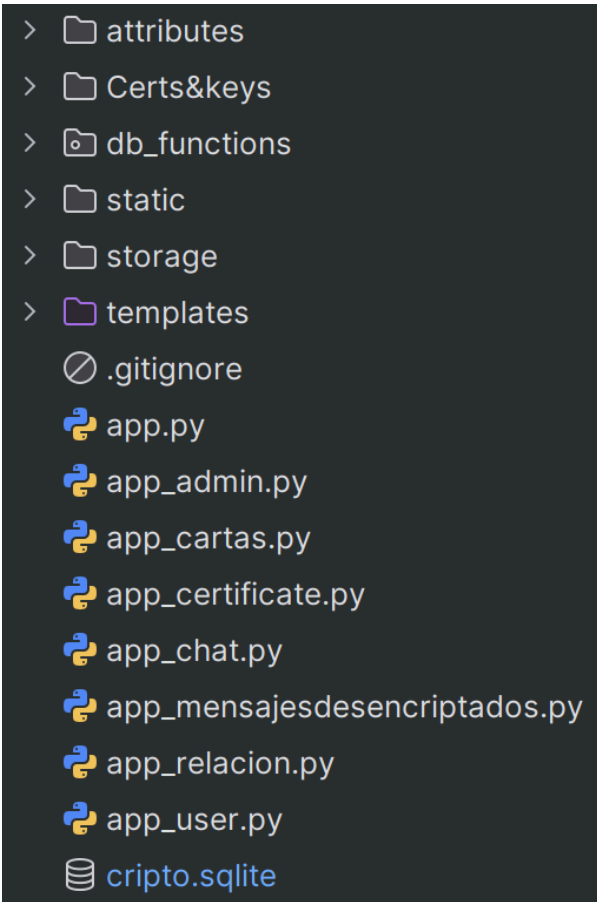
En la carpeta ‘attributes’ estarán todos los archivos que en la segunda entrega se encargará de comprobar los datos que un usuario introduzca por pantalla.

‘Certs&keys’ tiene los archivos referentes a los certificados.

‘db_functions’ tiene todas las funciones relativas al guardado y procesamiento de datos en la base de datos.

‘static’ y ‘templates’ tendrán todos los archivos relacionados con el HTML y el CSS necesarios para que la aplicación se vea correctamente.

Por último hay una serie de archivos .py que se encargan del funcionamiento de la aplicación divididos para que cada uno se encargue de un grupo de funciones en concreto.



```
> attributes
> Certs&keys
> db_functions
> static
> storage
> templates
> .gitignore
app.py
app_admin.py
app_cartas.py
app_certificate.py
app_chat.py
app_mensajesdesencryptados.py
app_relacion.py
app_user.py
cripto.sqlite
```

Firma:

1. ¿Para qué se utiliza la firma digital?

En nuestro caso, son las cartas las que se firma y el sistema el que firma, usamos la firma digital con el objetivo de comprobar que la carta enviada ha sido firmada por el sistema, asegurando que no se ha introducido de forma externa una carta en nuestro sistema.

De forma indirecta, los certificados usados están firmados y se comprueba su firma.

2. ¿Qué algoritmos ha utilizado y por qué?

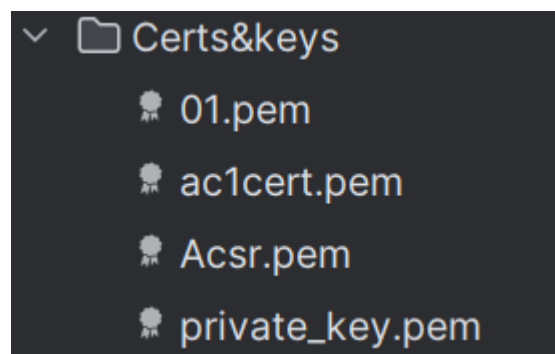
Para la generación de la clave privada del sistema, la usada para firmar, usamos RSA, por ser un estándar para la creación de claves que se involucran en la firma digital. Para la firma hemos usado el padding PSS por ser una opción más robusta que supera las limitaciones de PKCS1v15.

3. ¿Cómo se gestionan y almacenan las claves y las firmas?

A la hora de empezar el apartado de firma, generamos una clave privada mediante RSA y la guardamos serializada bajo una contraseña que está guardada en pepe.txt, junto a otras claves y contraseñas del sistema. Dado que del certificado 01.pem podemos obtener la clave pública del sistema, y es la única entidad que firma en nuestro código, hemos optado por no guardarla.

Tras tener la clave privada para la firma, generamos un CSR(Acsr.pem) y lo firmamos con esta clave, mediante OpenSSL, obtenemos el certificado con el que verificaremos la firma de nuestras cartas(01.pem), y el certificado de la entidad confiable, con el que verificaremos nuestro certificado y este.

Todos los certificados y la clave de esta parte del proyecto están almacenadas en la carpeta Certs&keys, por organización del proyecto.



Certificados:

1. ¿Cómo se generan los certificados de clave pública?

Los certificados de clave pública se generan por OpenSSL, todo empieza en el script de python `app_certificate.py`, donde se genera el CSR que se le pasará a una entidad confiable para que genere el certificado que usaremos para verificar la firma de las cartas.

2. ¿Qué jerarquía de autoridades de certificación se ha desplegado?

Por motivos de simplicidad y dado que el ejercicio requiere que aprendamos a manejar certificados, solo hay dos niveles en la jerarquía, la entidad confiable AC y nuestro sistema, por lo que solo disponemos de 2 certificados en nuestro sistema, el de la AC y el de la clave pública del sistema.

3. ¿Por qué ha escogido esta configuración y no otra?

Esta configuración se ha escogido por ser simple pero efectiva, necesitábamos aprender acerca de manejar certificados y necesitábamos de una entidad confiable que firmase nuestro certificado.

4. ¿Cómo se ha implementado?

Tras tener la clave privada, generamos el CSR, mediante OpenSSL generamos el certificado de clave pública del sistema, firmado por la AC y el certificado de la AC, también firmado por la AC. A la hora de verificar, verificamos primero el certificado de la AC (`ac1cert.pem`) con su clave pública, con la misma clave verificamos la firma del certificado del sistema (`01cert.pem`), si todo ha ido bien, verificamos la firma de la carta, y aseguramos que la carta fue firmada por la clave privada del sistema.

5. ¿En qué momento se utilizan los certificados y para qué?

Los certificados se usan para verificar la firma de la carta, se realiza cada vez que se añade una carta al sistema, tras guardar la carta en la base de datos. También se usan en la verificación de los propios certificados, pues de estos usamos la clave pública que contienen para verificar las firmas.

Complejidad y diseño:

Como hemos explicado en el apartado 2 de [Aplicación Web](#), hemos decidido organizar nuestro proyecto en diferentes subcarpetas para estructurar el código de manera más clara y ordenada. Esta organización permite agrupar funciones comunes en carpetas aisladas, evitando la repetición de código y promoviendo un diseño más limpio y alineado con las normas básicas de programación.

Reconocemos que esta estrategia puede aumentar la dificultad para localizar casos específicos debido a la cantidad de archivos generados. Para mitigar este inconveniente, hemos optado por establecer nombres intuitivos para las carpetas y archivos, facilitando la identificación de cada función, método o comprobación.

Además, para mejorar la comprensibilidad y elegancia del código, hemos adoptado SQLite como base de datos local. Esta solución es robusta y profesional, y permite gestionar de manera eficiente toda la información necesaria para el funcionamiento de nuestra aplicación.

Por último, hemos procurado diseñar un sistema elegante y fácil de entender, evitando complejidades innecesarias para que cualquier persona con conocimientos básicos de programación y Python pueda trabajar con el proyecto sin dificultad.

Mejoras:

En la realización de este proyecto se han realizado una serie de mejoras que explicaremos a continuación:

- I. **Almacenamiento en base de datos:** Inicialmente, para esta práctica utilizamos JSON como método de almacenamiento de la información. Sin embargo, posteriormente migramos todos los datos a una base de datos, en este caso SQLite, una solución local que permite almacenar, modificar y eliminar datos de manera más eficiente, elegante y profesional.
- II. **Validación de datos que introduce el usuario:** En diferentes secciones del programa hemos incluido validaciones exhaustivas para garantizar la seguridad de los datos de entrada:
 - A. **Registro e inicio de sesión:** Se verifica que los usuarios y contraseñas no contengan caracteres maliciosos.
 - B. **Envío de cartas:** Se controla que los caracteres permitidos sean los legales y válidos para escritura en las cajas de formulario.
 - C. **Envío de mensajes entre usuarios:** También se comprueba que únicamente se pueda ingresar un nombre de usuario válido y que los mensajes no incluyan caracteres maliciosos.
- III. **Rotación de claves:** Nuestro sistema permite establecer un rol de Administrador, el cual tiene la capacidad de realizar una renovación de claves. Este proceso actualiza todas las claves públicas existentes y cifra nuevamente los mensajes utilizando una nueva clave simétrica y un nuevo nonce.
- IV. **Captura de excepciones:** Debido a la cantidad de posibles errores que podrían surgir, hemos implementado un sistema de captura de excepciones. Los errores detectados se imprimen en la consola, proporcionando al usuario información detallada sobre lo ocurrido para facilitar su comprensión y solución.
- V. **Eliminación de mensajes:** Tanto el Administrador como el Técnico tienen la capacidad de eliminar mensajes entre usuarios que consideren inapropiados o que no respeten las normas básicas de decencia humana.
- VI. **Baneo de usuarios:** El Administrador y el Técnico también pueden banear o vetar usuarios. Esta acción elimina todos los mensajes enviados y recibidos por el usuario afectado y registra su correo electrónico en una lista de correos vetados, impidiéndole registrarse nuevamente con ese correo.
- VII. **Respuesta Papa Noel:** Finalmente, hemos implementado una funcionalidad que permite que, cada vez que un usuario envíe una carta, Papá Noel responda automáticamente al correo electrónico proporcionado en el formulario de envío.

Estas mejoras aumentan la seguridad de nuestro sistema al prevenir el ingreso de datos maliciosos, proteger la privacidad mediante la rotación de claves y reforzar el control con el baneo de usuarios y eliminación de mensajes inapropiados. Todo esto asegura un sistema confiable y seguro.

Declaración de uso de IA:

Para la realización de esta práctica se han utilizado varias IAs para algunos aspectos en concreto:

1. Se ha utilizado AI Assistant, la IA incorporada en Pycharm, principalmente para consultar dudas relacionadas con la parte de HTML, cómo transferir datos de un archivo HTML a otro, y cómo leer datos de un archivo JSON para mostrarlos por pantalla.
2. Por otro lado, se ha utilizado Copilot, la IA de Microsoft, principalmente para consultar ciertos errores que podían surgir y que nos explicara su significado.

Aparte de esos casos en concreto, la demás información utilizada para la realización de la práctica se ha obtenido de las clases de laboratorio, de la página web de ['cryptography'](#) y de diversos repositorios y guías encontradas en internet.